

Práctica 2: Análisis de Calidad Industrial (Industria 4.0)

Asignatura: Ciencia de Datos | Ingeniería en Sistemas Computacionales

1. Contexto Profesional

La empresa "TechManufacture" ha implementado sensores en sus líneas de ensamblaje. Se sospecha que las variaciones de temperatura y presión en las máquinas están afectando la calidad del producto final. Como Ingeniero de Datos, se te ha entregado un reporte de los últimos 500 lotes de producción para realizar un análisis estadístico exhaustivo.

2. Objetivo

Poner a prueba las habilidades de manipulación de datos con `dplyr`, cálculo de medidas estadísticas y visualización con `ggplot2` para generar interpretaciones de negocio.

3. Requerimientos

- Entorno: **RStudio**.
- Librerías: `tidyverse` (que incluye `dplyr` y `ggplot2`).
- Dataset: Se generará de forma sintética dentro del script.

4. Tareas a realizar

Fase A: Análisis Exploratorio (EDA)

1. Calcular la **media, mediana y desviación estándar** de la temperatura de operación.
2. Identificar si existen valores faltantes (`NA`) y aplicar una técnica de limpieza.
3. Determinar la **correlación de Pearson** entre la `temperatura` y la `tasa_error`.

Fase B: Agrupación y Segmentación

1. Comparar el rendimiento promedio (tasa de error) entre los turnos **Matutino** y **Vespertino**.
2. ¿Existe una diferencia significativa en la temperatura de operación entre turnos?

Fase C: Visualización e Interpretación

1. Crear un **Boxplot** para visualizar la dispersión de errores por turno (identificar *outliers*).

2. Generar un **Scatter Plot** (Gráfico de dispersión) con una línea de regresión para mostrar la relación entre temperatura y errores.

5. Entregable

Un archivo en formato **R Markdown (.Rmd)** o **Quarto (.qmd)** que incluya el código, las gráficas generadas y un párrafo de conclusión interpretando los resultados (¿Debería la empresa invertir en mejores sistemas de enfriamiento?).

```
# =====
```

```
# TechManufacture - Análisis Estadístico
```

```
# =====
```

```
# Librerías
```

```
import numpy as np
```

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
from scipy.stats import pearsonr, ttest_ind
```

```
# -----
```

```
# Generación del dataset sintético
```

```
# -----
```

```
np.random.seed(42)
```

```
n = 500
```

```
turnos = np.random.choice(  
    ["Matutino", "Vespertino"],  
    size=n,  
    p=[0.5, 0.5]  
)
```

```
temperatura = np.random.normal(  
    loc=75,  
    scale=5,  
    size=n  
)
```

```
presion = np.random.normal(  
    loc=100,  
    scale=8,  
    size=n  
)
```

Tasa de error influenciada por temperatura y presión

```
tasa_error = (  
    0.2 * (temperatura - 75)  
    + 0.05 * (presion - 100)
```

```

    + np.random.normal(0, 1.5, n)
)

# Crear DataFrame
df = pd.DataFrame({
    "turno": turnos,
    "temperatura": temperatura,
    "presion": presion,
    "tasa_error": tasa_error
})

# -----

# Introducir valores faltantes (NA)

# -----

indices_na = np.random.choice(df.index, size=15, replace=False)
df.loc[indices_na, "temperatura"] = np.nan

print("Primeras filas del dataset:")
print(df.head())

# =====

# FASE A: ANÁLISIS EXPLORATORIO (EDA)

```

```
# =====
```

```
print("\n" + "="*50)
```

```
print("FASE A: EDA")
```

```
print("="*50)
```

```
# 1. Media, mediana y desviación estándar
```

```
media_temp = df["temperatura"].mean()
```

```
mediana_temp = df["temperatura"].median()
```

```
desv_temp = df["temperatura"].std()
```

```
print(f"\nMedia temperatura: {media_temp:.2f}")
```

```
print(f"Mediana temperatura: {mediana_temp:.2f}")
```

```
print(f"Desviación estándar: {desv_temp:.2f}")
```

```
# 2. Identificar valores faltantes
```

```
print("\nValores faltantes:")
```

```
print(df.isna().sum())
```

```
# Limpieza mediante imputación con la media
```

```
df["temperatura"] = df["temperatura"].fillna(
```

```
    df["temperatura"].mean()
```

```
)
```

```
print("\nValores faltantes después de limpiar:")
```

```
print(df.isna().sum())
```

```
# 3. Correlación de Pearson
```

```
corr, p_value = pearsonr(
```

```
    df["temperatura"],
```

```
    df["tasa_error"]
```

```
)
```

```
print("\nCorrelación de Pearson:")
```

```
print(f'r = {corr:.4f}')
```

```
print(f'p-value = {p_value:.4f}')
```

```
# =====
```

```
# FASE B: AGRUPACIÓN Y SEGMENTACIÓN
```

```
# =====
```

```
print("\n" + "="*50)
```

```
print("FASE B: AGRUPACIÓN")
```

```
print("="*50)
```

```
# 1. Comparar tasa de error promedio por turno
```

```

promedio_error = (
    df.groupby("turno")["tasa_error"]
    .mean()
    .reset_index()
)

print("\nPromedio de tasa de error por turno:")
print(promedio_error)

# 2. Diferencia significativa en temperatura
temp_matutino = df[df["turno"] == "Matutino"]["temperatura"]
temp_vespertino = df[df["turno"] == "Vespertino"]["temperatura"]

t_stat, p_temp = ttest_ind(
    temp_matutino,
    temp_vespertino,
    equal_var=False
)

print("\nPrueba t para temperatura entre turnos")
print(f"Estadístico t = {t_stat:.4f}")
print(f"p-value = {p_temp:.4f}")

```

```
if p_temp < 0.05:
    print("Existe diferencia significativa entre turnos.")
else:
    print("No existe diferencia significativa entre turnos.")
```

```
# =====
```

```
# FASE C: VISUALIZACIÓN
```

```
# =====
```

```
sns.set_style("whitegrid")
```

```
# -----
```

```
# Boxplot
```

```
# -----
```

```
plt.figure(figsize=(8,5))
```

```
sns.boxplot(
    data=df,
    x="turno",
    y="tasa_error"
)
```

```
plt.title("Distribución de la Tasa de Error por Turno")
```



```
plt.xlabel("Turno")  
plt.ylabel("Tasa de Error")  
plt.show()
```

```
# -----
```

```
# Scatter Plot + Regresión
```

```
# -----
```

```
plt.figure(figsize=(8,5))
```

```
sns.regplot(  
    data=df,  
    x="temperatura",  
    y="tasa_error",  
    scatter_kws={"alpha":0.6}  
)
```

```
plt.title("Relación entre Temperatura y Tasa de Error")
```

```
plt.xlabel("Temperatura")
```

```
plt.ylabel("Tasa de Error")
```

```
plt.show()
```

```
# =====
```

```
# CONCLUSIÓN AUTOMÁTICA
```

```
# =====
```

```
print("\n" + "="*50)
```

```
print("CONCLUSIÓN")
```

```
print("="*50)
```

```
if corr > 0.3:
```

```
    print(
```

```
        "Se observa una correlación positiva moderada o fuerte "
```

```
        "entre la temperatura y la tasa de error. "
```

```
        "La empresa debería considerar invertir en sistemas "
```

```
        "de enfriamiento para reducir defectos."
```

```
    )
```

```
else:
```

```
    print(
```

```
        "La relación entre temperatura y errores es débil. "
```

```
        "Se recomienda investigar otros factores además de "
```

```
        "la temperatura antes de realizar inversiones."
```

```
    )
```

```
In [2]: %runfile 'C:/Users/liclo/Documents/Sin título0.py' --wdir
Primeras filas del dataset:
```

	turno	temperatura	presion	tasa_error
0	Matutino	76.708780	104.671425	2.527939
1	Vespertino	84.380854	97.125663	4.074721
2	Vespertino	79.752119	104.725239	1.234692
3	Vespertino	72.115482	108.869629	-1.263549
4	Matutino	70.507927	106.563857	0.119736

```
=====
FASE A: EDA
=====
```

```
Media temperatura: 75.09
Mediana temperatura: 74.92
Desviación estándar: 5.00
```

```
Valores faltantes:
```

```
turno      0
temperatura 15
presion     0
tasa_error  0
dtype: int64
```

```
Valores faltantes después de limpiar:
```

```
turno      0
temperatura 0
presion     0
tasa_error  0
dtype: int64
```

```
Correlación de Pearson:
```

```
r = 0.5076
p-value = 0.0000
```

```
=====
FASE B: AGRUPACIÓN
=====
```

```
Promedio de tasa de error por turno:
```

	turno	tasa_error
0	Matutino	0.004752
1	Vespertino	0.276144

```
Prueba t para temperatura entre turnos
```

```
Estadístico t = -1.3275
```

```
p-value = 0.1850
```

```
No existe diferencia significativa entre turnos.
```

Práctica 3: Análisis de Eficiencia en Logística Global

Asignatura: Ciencia de Datos | Ingeniería en Sistemas Computacionales

1. Contexto Profesional

La empresa "Global-Logistics ISC" utiliza dos modalidades de transporte para sus envíos nacionales: **Terrestre** y **Aéreo**. La gerencia de operaciones ha notado retrasos y sospecha que el tiempo de entrega no solo depende de la distancia, sino de ineficiencias en la modalidad de transporte bajo ciertas condiciones climáticas. Tu misión es analizar los datos de los últimos 600 envíos para determinar dónde están los cuellos de botella.

2. Objetivo

Validar de forma independiente la capacidad de limpiar datos, realizar análisis exploratorio, agrupar información y aplicar pruebas de hipótesis para la toma de decisiones basada en datos.

3. Requerimientos

- Entorno: **RStudio**.
- Librerías: `tidyverse` (`dplyr`, `ggplot2`).
- Dataset: Simulación de 600 registros generada en el script.

4. Tareas a realizar por el alumno

Fase 1: Calidad de Datos

1. **Limpieza:** El dataset contiene valores NA en la columna `distancia_km` debido a fallas en el GPS. Imputa estos valores usando la **mediana**.
2. **Descriptivos:** Calcula la media y desviación estándar del `tiempo_entrega_hrs`.

Fase 2: Análisis de Relaciones

1. **Correlación:** Calcula el coeficiente de Pearson entre `distancia_km` y `tiempo_entrega_hrs`. ¿Es la relación tan fuerte como se esperaba?

Fase 3: Comparativa de Modalidades

1. **Agrupación:** Agrupa por `tipo_transporte` y obtén el promedio de tiempo de entrega y el costo promedio.
2. **Prueba de Hipótesis:** Aplica un **t-test** para determinar si existe una diferencia significativa entre el tiempo de entrega del transporte **Terrestre** vs **Aéreo**.

Fase 4: Visualización

1. Genera un **Gráfico de Dispersión** que muestre la relación entre distancia y tiempo, diferenciando los puntos por modalidad de transporte.
2. Crea un **Boxplot** para comparar los tiempos de entrega entre modalidades e identificar valores atípicos (envíos extremadamente retrasados).

5. Entregable

Un reporte técnico en **R Markdown** donde se presente el análisis, las gráficas y una recomendación final: ¿Debería la empresa migrar más envíos al transporte aéreo para mejorar la satisfacción del cliente?

```
# =====
```

```
# Global-Logistics ISC
```

```
# Análisis Estadístico de Envíos Nacionales
```

```
# =====
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from scipy.stats import pearsonr, ttest_ind
```

```
# -----
```

```
# FASE 0: Generación del Dataset (600 registros)
```

```
# -----
```

```
np.random.seed(42)
```

```
n = 600
```

```
# Modalidades de transporte
```

```
tipo_transporte = np.random.choice(  
    ['Terrestre', 'Aéreo'],  
    size=n,  
    p=[0.65, 0.35]  
)
```

```
# Condiciones climáticas
```

```
clima = np.random.choice(  
    ['Normal', 'Lluvia', 'Tormenta'],  
    size=n,  
    p=[0.6, 0.3, 0.1]  
)
```

```
# Distancias simuladas
```

```
distancia_km = np.random.randint(50, 2500, n)
```

```
# Introducir valores faltantes (NA)
```

```
indices_na = np.random.choice(n, size=30, replace=False)
```

```
distancia_km = distancia_km.astype(float)
```

```
distancia_km[indices_na] = np.nan
```

```
# Tiempo de entrega según modalidad
```

```
tiempo_entrega = []
```

```
for i in range(n):
```

```
    dist = distancia_km[i]
```

```
    # Usar distancia temporal para simulación
```

```
    if np.isnan(dist):
```

```
        dist = np.random.randint(50, 2500)
```

```
    if tipo_transporte[i] == 'Terrestre':
```

```
        tiempo = dist / 60 + np.random.normal(5, 2)
```

```
    else:
```

```
        tiempo = dist / 500 + np.random.normal(2, 0.8)
```

```
    # Efecto del clima
```

```
    if clima[i] == 'Lluvia':
```

```
        tiempo += np.random.uniform(1, 4)
```

```

elif clima[i] == 'Tormenta':

    tiempo += np.random.uniform(3, 8)


tiempo_entrega.append(tiempo)


tiempo_entrega = np.array(tiempo_entrega)


# Costos simulados

costo = []


for i in range(n):

    dist = distancia_km[i]


    if np.isnan(dist):

        dist = 1000


    if tipo_transporte[i] == 'Terrestre':

        costo.append(dist * 0.80 + np.random.normal(50, 20))

    else:

        costo.append(dist * 1.50 + np.random.normal(100, 40))


# Crear DataFrame

df = pd.DataFrame({

```



```
'tipo_transporte': tipo_transporte,  
  
'clima': clima,  
  
'distancia_km': distancia_km,  
  
'tiempo_entrega_hrs': tiempo_entrega,  
  
'costo_envio': costo  
  
})
```

```
print("\nPrimeros registros:")
```

```
print(df.head())
```

```
# -----
```

```
# FASE 1: Calidad de Datos
```

```
# -----
```

```
print("\n=====")
```

```
print("FASE 1: CALIDAD DE DATOS")
```

```
print("=====")
```

```
# 1. Imputación por mediana
```

```
mediana_distancia = df['distancia_km'].median()
```

```
df['distancia_km'].fillna(  
    mediana_distancia,
```

```

        inplace=True
    )

    print(f"\nMediana utilizada: {mediana_distancia:.2f} km")

```

2. Estadísticos descriptivos

```

media_tiempo = df['tiempo_entrega_hrs'].mean()

desv_tiempo = df['tiempo_entrega_hrs'].std()

```

```

print(f"\nMedia tiempo entrega: {media_tiempo:.2f} hrs")

print(f"Desviación estándar: {desv_tiempo:.2f} hrs")

```

```

# -----

```

FASE 2: Correlación

```

# -----

```

```

print("\n=====")

print("FASE 2: CORRELACIÓN")

print("=====")

```

```

corr, p_valor = pearsonr(

    df['distancia_km'],

    df['tiempo_entrega_hrs']

```

)

```
print(f"\nCoeficiente Pearson: {corr:.4f}")
```

```
print(f"p-value: {p_valor:.8f}")
```

```
# -----
```

```
# FASE 3: Comparación de Modalidades
```

```
# -----
```

```
print("\n=====")
```

```
print("FASE 3: COMPARACIÓN")
```

```
print("=====")
```

```
agrupado = df.groupby(
```

```
    'tipo_transporte'
```

```
).agg({
```

```
    'tiempo_entrega_hrs': 'mean',
```

```
    'costo_envio': 'mean'
```

```
})
```

```
print("\nPromedios por modalidad:")
```

```
print(agrupado)
```

```
# T-Test

terrestre = df[

    df['tipo_transporte'] == 'Terrestre'

][['tiempo_entrega_hrs']]

aereo = df[

    df['tipo_transporte'] == 'Aéreo'

][['tiempo_entrega_hrs']]

t_stat, p_ttest = ttest_ind(

    terrestre,

    aereo,

    equal_var=False

)

print("\nResultados T-Test")

print(f"Estadístico t: {t_stat:.4f}")

print(f"p-value: {p_ttest:.8f}")

if p_ttest < 0.05:

    print("Existe diferencia significativa entre modalidades.")

else:

    print("No existe diferencia significativa.")
```

```
# -----
```

```
# FASE 4: Visualizaciones
```

```
# -----
```

```
sns.set_style("whitegrid")
```

```
# Gráfico de dispersión
```

```
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(
```

```
    data=df,
```

```
    x='distancia_km',
```

```
    y='tiempo_entrega_hrs',
```

```
    hue='tipo_transporte'
```

```
)
```

```
plt.title(
```

```
    'Distancia vs Tiempo de Entrega'
```

```
)
```

```
plt.xlabel('Distancia (km)')
```

```
plt.ylabel('Tiempo de Entrega (hrs)')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Boxplot
```

```
plt.figure(figsize=(8, 6))
```

```
sns.boxplot(
```

```
    data=df,
```

```
    x='tipo_transporte',
```

```
    y='tiempo_entrega_hrs'
```

```
)
```

```
plt.title(
```

```
    'Comparación de Tiempos de Entrega'
```

```
)
```

```
plt.xlabel('Modalidad')
```

```
plt.ylabel('Tiempo de Entrega (hrs)')
```

```
plt.tight_layout()
```

```
plt.show()
```

```

# -----

# CONCLUSIÓN AUTOMÁTICA

# -----

print("\n=====")

print("RECOMENDACIÓN")

print("=====")


promedio_terrestre = terrestre.mean()

promedio_aereo = aereo.mean()


print(f"\nTiempo promedio terrestre: {promedio_terrestre:.2f} hrs")

print(f"Tiempo promedio aéreo: {promedio_aereo:.2f} hrs")


if p_ttest < 0.05 and promedio_aereo < promedio_terrestre:

    print(

        "\nRECOMENDACIÓN: "

        "Los datos sugieren migrar una mayor proporción "

        "de envíos al transporte aéreo para reducir los "

        "tiempos de entrega y mejorar la satisfacción del cliente."

    )

else:

    print(

```

"\nRECOMENDACIÓN: "

"No existe evidencia estadística suficiente para justificar "

"una migración masiva al transporte aéreo."

)

```
Primeros registros:
  tipo_transporte  clima  distancia_km  tiempo_entrega_hrs  costo_envio
0      Terrestre  Normal    2478.0         46.871970    2043.377677
1        Aéreo  Normal    1080.0         3.816926    1709.801768
2        Aéreo  Normal     78.0         1.093651     211.981830
3      Terrestre  Normal     956.0        23.099691     821.357590
4      Terrestre  Normal    2176.0        42.740182    1792.517860
```

```
=====
FASE 1: CALIDAD DE DATOS
=====

Mediana utilizada: 1268.50 km

Media tiempo entrega: 19.68 hrs
Desviación estándar: 14.28 hrs
```

```
=====
FASE 2: CORRELACIÓN
=====

Coeficiente Pearson: 0.5455
p-value: 0.00000000
```

```
=====
FASE 3: COMPARACIÓN
=====

Promedios por modalidad:
              tiempo_entrega_hrs  costo_envio
tipo_transporte
Aéreo              5.872584    2025.050318
Terrestre          27.121245    1026.484974
```



```
Resultados T-Test
Estadístico t: 32.7900
p-value: 0.00000000
Existe diferencia significativa entre modalidades.
c:\users\liclo\documents\sin título1.py:112: FutureWarning: A value is trying to be set on a copy of
a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate
object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value},
inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the
original object.
```

```
df['distancia_km'].fillna(
=====
RECOMENDACIÓN
=====

Tiempo promedio terrestre: 27.12 hrs
Tiempo promedio aéreo: 5.87 hrs

RECOMENDACIÓN: Los datos sugieren migrar una mayor proporción de envíos al transporte aéreo para
reducir los tiempos de entrega y mejorar la satisfacción del cliente.
```

Práctica 4: Reducción de Dimensionalidad en Monitoreo Industrial

Asignatura: Ciencia de Datos | Unidad 2: Tratamiento de Datos

1. Contexto Profesional

La planta de fundición "Metal-Tech" ha instalado 12 sensores en una caldera crítica. Los sensores miden temperatura, presión, vibración, flujo de gas, emisiones de CO₂, entre otros. El equipo de TI reporta que procesar las 12 señales en tiempo real satura el ancho de banda del PLC. Tu tarea es aplicar **PCA** para reducir estas 12 variables a los componentes principales que capturen al menos el **85% de la varianza**, permitiendo un monitoreo eficiente.

2. Objetivo

Aplicar el algoritmo PCA en R para identificar variables redundantes y reducir la dimensionalidad de un sistema de sensores industriales.

3. Requerimientos

- Entorno: **RStudio**.
- Librería: `tidyverse`, `factoextra` (opcional para mejores gráficos).

4. Instrucciones para el Estudiante

Fase 1: Preparación

1. Cargar el dataset de sensores (generado en el script).
2. Observar la correlación inicial entre sensores. ¿Hay sensores que miden "lo mismo"?

Fase 2: Ejecución de PCA

1. Utilizar la función `prcomp()`. **Importante:** Asegúrate de estandarizar los datos (`scale. = TRUE`).
2. Analizar el objeto resultante con `summary()`.

Fase 3: Selección de Componentes

1. Generar el **Scree Plot**.

2. Determinar cuántos componentes son necesarios para alcanzar el 85% de varianza acumulada.
3. Identificar qué sensores originales tienen más peso ("loadings") en el primer componente principal.

Fase 4: Visualización

1. Crear un **Biplot** para visualizar cómo se agrupan los lotes de producción en el nuevo espacio de dimensiones reducidas.

```
# =====
```

```
# PCA EN SENSORES INDUSTRIALES - METAL-TECH
```

```
# =====
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA
```

```
# -----
```

```
# FASE 1: GENERACIÓN DEL DATASET
```

```
# -----
```

```
np.random.seed(42)
```

```
n_muestras = 500
```

```

# Variables base para crear correlaciones

temperatura_base = np.random.normal(800, 40, n_muestras)

presion_base = np.random.normal(100, 10, n_muestras)

vibracion_base = np.random.normal(20, 3, n_muestras)


# Sensores correlacionados

datos = pd.DataFrame({

    'Temp_1': temperatura_base + np.random.normal(0, 5, n_muestras),

    'Temp_2': temperatura_base + np.random.normal(0, 4, n_muestras),

    'Temp_3': temperatura_base + np.random.normal(0, 6, n_muestras),


    'Presion_1': presion_base + np.random.normal(0, 2, n_muestras),

    'Presion_2': presion_base + np.random.normal(0, 2, n_muestras),

    'Presion_3': presion_base + np.random.normal(0, 3, n_muestras),


    'Vibracion_1': vibracion_base + np.random.normal(0, 1, n_muestras),

    'Vibracion_2': vibracion_base + np.random.normal(0, 1, n_muestras),


    'Flujo_Gas': temperatura_base * 0.05 + np.random.normal(50, 3, n_muestras),

    'CO2': temperatura_base * 0.03 + np.random.normal(20, 2, n_muestras),


    'Nivel_Agua': np.random.normal(60, 8, n_muestras),

    'Consumo_Energia': temperatura_base * 0.08 +

```

```

        presion_base * 0.5 +

        np.random.normal(0, 5, n_muestras)

    })

print("\nPrimeras filas del dataset:")

print(datos.head())


# -----
# MATRIZ DE CORRELACIÓN
# -----

correlacion = datos.corr()

print("\nMatriz de correlación:")

print(correlacion.round(2))


plt.figure(figsize=(10,8))

plt.imshow(correlacion, cmap='coolwarm', aspect='auto')

plt.colorbar(label='Correlación')

plt.xticks(range(len(datos.columns)), datos.columns, rotation=90)

plt.yticks(range(len(datos.columns)), datos.columns)

plt.title("Matriz de Correlación de Sensores")

plt.tight_layout()

```

```
plt.show()
```

```
# -----
```

```
# FASE 2: ESTANDARIZACIÓN
```

```
# -----
```

```
scaler = StandardScaler()
```

```
datos_escalados = scaler.fit_transform(datos)
```

```
# -----
```

```
# PCA
```

```
# -----
```

```
pca = PCA()
```

```
componentes = pca.fit_transform(datos_escalados)
```

```
# Resumen de varianza explicada
```

```
varianza = pca.explained_variance_ratio_
```

```
varianza_acumulada = np.cumsum(varianza)
```

```
resumen = pd.DataFrame({
```

```
    'Componente': range(1, len(varianza)+1),
```

```
    'Varianza Explicada': varianza,
```

```

        'Varianza Acumulada': varianza_acumulada
    })

print("\nResumen PCA:")
print(resumen)

# -----
# FASE 3: SELECCIÓN DE COMPONENTES
# -----

n_componentes = np.argmax(varianza_acumulada >= 0.85) + 1

print("\nNúmero de componentes necesarios para alcanzar 85% de varianza:")
print(n_componentes)

# -----
# SCREE PLOT
# -----

plt.figure(figsize=(8,5))
plt.plot(range(1, len(varianza)+1),
        varianza,
        marker='o',

```

```
linestyle='--')
```

```
plt.xlabel("Componente Principal")
```

```
plt.ylabel("Varianza Explicada")
```

```
plt.title("Scree Plot")
```

```
plt.grid(True)
```

```
plt.show()
```

```
# -----
```

```
# LOADINGS DEL PRIMER COMPONENTE
```

```
# -----
```

```
loadings = pd.DataFrame(
```

```
    pca.components_.T,
```

```
    columns=[f'PC {i+1}' for i in range(len(datos.columns))],
```

```
    index=datos.columns
```

```
)
```

```
print("\nLoadings del primer componente principal (PC1):")
```

```
print(loadings['PC1'].sort_values(key=abs, ascending=False))
```

```
# Top sensores más influyentes
```

```
print("\nSensores con mayor peso en PC1:")
```



```
print(loadings['PC1'].abs().sort_values(ascending=False).head(5))
```

```
# -----
```

```
# PCA REDUCIDO AL 85%
```

```
# -----
```

```
pca_reducido = PCA(n_components=n_componentes)
```

```
datos_pca = pca_reducido.fit_transform(datos_escalados)
```

```
print(f"\nDimensión original: {datos.shape[1]} variables")
```

```
print(f"Dimensión reducida: {n_componentes} componentes")
```

```
# -----
```

```
# FASE 4: BILOT
```

```
# -----
```

```
plt.figure(figsize=(10,8))
```

```
# Observaciones
```

```
plt.scatter(
```

```
    componentes[:, 0],
```

```
    componentes[:, 1],
```

```
    alpha=0.5
```

```
)
```

```
# Vectores de variables
```

```
for i, variable in enumerate(datos.columns):
```

```
    plt.arrow(
```

```
        0, 0,
```

```
        pca.components_[0, i] * 5,
```

```
        pca.components_[1, i] * 5,
```

```
        head_width=0.05
```

```
    )
```

```
    plt.text(
```

```
        pca.components_[0, i] * 5.2,
```

```
        pca.components_[1, i] * 5.2,
```

```
        variable
```

```
    )
```

```
plt.xlabel(
```

```
    f'PC1 ({varianza[0]*100:.2f}% varianza)'
```

```
)
```

```
plt.ylabel(
```

```
    f'PC2 ({varianza[1]*100:.2f}% varianza)'
```

```
)
```

```

plt.title("Biplot PCA - Sensores Metal-Tech")

plt.grid(True)

plt.show()

# -----

# INTERPRETACIÓN AUTOMÁTICA

# -----

print("\nINTERPRETACIÓN:")

if n_componentes < datos.shape[1]:

    reduccion = (1 - n_componentes/datos.shape[1]) * 100

    print(f"Se logró una reducción del {reduccion:.2f}% "

          f"en la dimensionalidad.")

print("Los sensores con cargas altas en PC1 son los que más")

print("contribuyen al comportamiento global del sistema.")

print("Sensores altamente correlacionados pueden considerarse")

print("redundantes para monitoreo en tiempo real.")

```

```
In [4]: %runfile 'C:/Users/liclo/Documents/Sin título2.py' --wdir
```

Primeras filas del dataset:

	Temp_1	Temp_2	Temp_3	...	CO2	Nivel_Agua	Consumo_Energia
0	823.760372	817.167853	823.292244	...	42.367894	69.818191	124.146298
1	791.713499	793.891353	800.278536	...	42.572221	50.318716	114.216233
2	821.816547	822.737862	823.919691	...	42.893106	63.508634	112.653494
3	860.904322	859.689348	857.247774	...	44.731644	49.039787	120.519921
4	789.782942	783.059406	784.122962	...	43.290715	68.523010	113.534785

Matriz de correlación:

	Temp_1	Temp_2	Temp_3	...	CO2	Nivel_Agua	Consumo_Energia
Temp_1	1.00	0.99	0.98	...	0.51	-0.02	0.42
Temp_2	0.99	1.00	0.98	...	0.51	-0.03	0.42
Temp_3	0.98	0.98	1.00	...	0.50	-0.03	0.42
Presion_1	-0.07	-0.07	-0.06	...	-0.03	-0.09	0.56
Presion_2	-0.08	-0.07	-0.07	...	-0.04	-0.06	0.56
Presion_3	-0.09	-0.08	-0.08	...	-0.04	-0.07	0.56
Vibracion_1	-0.05	-0.05	-0.05	...	-0.05	0.04	0.04
Vibracion_2	-0.06	-0.06	-0.06	...	-0.05	0.02	0.05
Flujo_Gas	0.55	0.56	0.55	...	0.31	0.01	0.21
CO2	0.51	0.51	0.50	...	1.00	-0.08	0.23
Nivel_Agua	-0.02	-0.03	-0.03	...	-0.08	1.00	-0.02
Consumo_Energia	0.42	0.42	0.42	...	0.23	-0.02	1.00

Reporte Técnico: Reducción de Dimensionalidad en Monitoreo Industrial

Unidad 2: Tratamiento de Datos - Ciencia de Datos"

1. Introducción y Objetivo

Instrucciones: Describe brevemente el problema de la planta "Metal-Tech". ¿Por qué es necesario reducir las dimensiones de los 12 sensores? ¿Qué beneficios técnicos y económicos aporta el PCA en este escenario?

2. Metodología y Carga de Datos

En esta sección se inicializa el entorno y se cargan las señales de los sensores.

```
# Generación del dataset de sensores (Código proporcionado en la
práctica)
set.seed(42)
n <- 200
temp1 <- rnorm(n, 100, 5)
temp2 <- temp1 + rnorm(n, 0, 1)
presion1 <- rnorm(n, 50, 10)
vibracion <- (temp1 * 0.5) + rnorm(n, 0, 2)

datos_sensores <- data.frame(
  temp_nucleo = temp1, temp_escape = temp2, presion_interna = presion1,
  vibracion_motor = vibracion, flujo_gas = rnorm(n, 20, 2),
  oxigeno = rnorm(n, 15, 1), co2 = temp1 * 0.2 + rnorm(n, 0, 0.5),
  humedad = runif(n, 30, 40), voltaje = rnorm(n, 220, 2),
  corriente = rnorm(n, 15, 0.5), ruido_db = vibracion * 1.2 + rnorm(n, 0,
1),
  eficiencia = 100 - (temp1 * 0.1)
)

head(datos_sensores)
```

3. Ejecución del Algoritmo PCA

Se aplica la función `prcomp` asegurando la estandarización de las variables.

```
# Ejecución del PCA con escalamiento
pca_modelo <- prcomp(datos_sensores, center = TRUE, scale. = TRUE)

# Resumen de varianza
summary(pca_modelo)
```

4. Análisis de Resultados

4.1 Selección de Componentes (Scree Plot)

Tarea: Genera la gráfica de sedimentación e identifica el "codo". Justifica cuántos componentes seleccionarías para cumplir con el objetivo del 85% de varianza.

```
# Código para generar el gráfico de sedimentación
plot(pca_modelo, type = "l", main = "Gráfico de Sedimentación (Scree Plot)")
abline(h = 1, col="red", lty=2) # Criterio de Kaiser
```

Análisis del estudiante: ¿Cuántos componentes explican el 85%? ¿Por qué elegiste ese número?

4.2 Cargas de los Componentes (Loadings)

Tarea: Observa qué variables originales tienen más peso en los primeros dos componentes.

```
# Ver la rotación (loadings)
round(pca_modelo$rotation[, 1:2], 3)
```

Interpretación: ¿Qué representan físicamente el PC1 y el PC2? (Ejemplo: PC1 parece representar el 'Estado Térmico' del sistema).

4.3 Visualización del Biplot

```
# Visualización de variables y observaciones
biplot(pca_modelo, cex = 0.7, main = "Biplot de Sensores Industriales")
```

5. Conclusiones y Recomendaciones

Instrucciones: Basado en tu análisis, responde a la gerencia de TI:

1. ¿A cuántas variables podemos reducir el monitoreo sin perder información crítica?
2. ¿Cuáles sensores son redundantes y podrían dejar de enviarse por el bus de datos?
3. ¿Cómo impacta esta reducción en la eficiencia del sistema de monitoreo en tiempo real?

=====

PCA PARA SENSORES INDUSTRIALES - PLANTA METAL-TECH

=====

Librerías

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA


# -----

# 1. GENERACIÓN DEL DATASET

# -----


np.random.seed(42)


n = 200


temp1 = np.random.normal(100, 5, n)
temp2 = temp1 + np.random.normal(0, 1, n)


presion1 = np.random.normal(50, 10, n)


vibracion = (temp1 * 0.5) + np.random.normal(0, 2, n)


datos_sensores = pd.DataFrame({
```

```
'temp_nucleo': temp1,  
'temp_escape': temp2,  
'presion_interna': presion1,  
'vibracion_motor': vibracion,  
'flujo_gas': np.random.normal(20, 2, n),  
'oxigeno': np.random.normal(15, 1, n),  
'co2': temp1 * 0.2 + np.random.normal(0, 0.5, n),  
'humedad': np.random.uniform(30, 40, n),  
'voltaje': np.random.normal(220, 2, n),  
'corriente': np.random.normal(15, 0.5, n),  
'ruido_db': vibracion * 1.2 + np.random.normal(0, 1, n),  
'eficiencia': 100 - (temp1 * 0.1)  
})
```

```
print("\nPrimeras filas del dataset:")
```

```
print(datos_sensores.head())
```

```
# -----
```

```
# 2. ESTANDARIZACIÓN DE DATOS
```

```
# -----
```

```
escalador = StandardScaler()
```



```
datos_escalados = escalador.fit_transform(datos_sensores)
```

```
# -----
```

```
# 3. EJECUCIÓN DEL PCA
```

```
# -----
```

```
pca = PCA()
```

```
componentes = pca.fit_transform(datos_escalados)
```

```
# -----
```

```
# 4. VARIANZA EXPLICADA
```

```
# -----
```

```
varianza = pca.explained_variance_ratio_
```

```
varianza_acumulada = np.cumsum(varianza)
```

```
tabla_varianza = pd.DataFrame({  
    'Componente': np.arange(1, len(varianza)+1),  
    'Varianza Explicada (%)': np.round(varianza*100, 2),  
    'Varianza Acumulada (%)': np.round(varianza_acumulada*100, 2)  
})
```

```

print("\nResumen de Varianza:")

print(tabla_varianza)


# Encontrar número de componentes para 85%
n_componentes_85 = np.argmax(varianza_acumulada >= 0.85) + 1


print("\nComponentes necesarios para explicar al menos 85% de la varianza:")
print(n_componentes_85)


# -----

# 5. SCREE PLOT

# -----


plt.figure(figsize=(8,5))


plt.plot(
    range(1, len(varianza)+1),
    pca.explained_variance_,
    marker='o'
)


plt.axhline(y=1, color='red', linestyle='--')

```

```
plt.xlabel("Componentes Principales")
```

```
plt.ylabel("Autovalor")
```

```
plt.title("Scree Plot - Sensores Industriales")
```

```
plt.grid(True)
```

```
plt.show()
```

```
# -----
```

```
# 6. LOADINGS (PESOS DE VARIABLES)
```

```
# -----
```

```
loadings = pd.DataFrame(
```

```
    pca.components_.T,
```

```
    columns=[f'PC {i+1}' for i in range(len(datos_sensores.columns))],
```

```
    index=datos_sensores.columns
```

```
)
```

```
print("\nLoadings de PC1 y PC2:")
```

```
print(loadings[['PC1', 'PC2']].round(3))
```

```
# Variables más influyentes
```

```
print("\nVariables más importantes en PC1:")
```

```
print(loadings['PC1'].abs().sort_values(ascending=False).head())
```

```
print("\nVariables más importantes en PC2:")  
  
print(loadings['PC2'].abs().sort_values(ascending=False).head())
```

```
# -----
```

```
# 7. BILOT
```

```
# -----
```

```
plt.figure(figsize=(10,8))
```

```
scores = componentes[:, :2]
```

```
plt.scatter(  
    scores[:,0],  
    scores[:,1],  
    alpha=0.5  
)
```

```
for i, variable in enumerate(datos_sensores.columns):
```

```
    plt.arrow(  
        0,  
        0,
```

```
        loadings.iloc[i,0]*5,  
        loadings.iloc[i,1]*5,  
        head_width=0.05  
    )
```

```
plt.text(  
    loadings.iloc[i,0]*5.2,  
    loadings.iloc[i,1]*5.2,  
    variable  
)
```

```
plt.xlabel("PC1")  
plt.ylabel("PC2")  
plt.title("Biplot de Sensores Industriales")  
plt.grid(True)
```

```
plt.show()
```

```
# -----
```

```
# 8. MATRIZ DE CORRELACIÓN
```

```
# -----
```

```
correlacion = datos_sensores.corr()
```

```

print("\nMatriz de correlación:")

print(correlacion.round(2))

# Detectar sensores redundantes

print("\nPosibles sensores redundantes (|correlación| > 0.85):")

for i in range(len(correlacion.columns)):
    for j in range(i+1, len(correlacion.columns)):

        corr = correlacion.iloc[i,j]

        if abs(corr) > 0.85:
            print(
                correlacion.columns[i],
                "<-->",
                correlacion.columns[j],
                f"(corr={corr:.3f})"
            )

# -----

# 9. CONCLUSIONES AUTOMÁTICAS

# -----

```

```
print("\n" + "="*60)
```

```
print("CONCLUSIONES")
```

```
print("="*60)
```

```
print(
```

```
    f"\n1. El sistema puede reducirse de 12 sensores "
```

```
    f"a aproximadamente {n_componentes_85} componentes "
```

```
    f"manteniendo más del 85% de la información."
```

```
)
```

```
print(
```

```
    "\n2. Sensores como temp_nucleo, temp_escape, "
```

```
    "co2, vibracion_motor y ruido_db probablemente "
```

```
    "presentan redundancia debido a su alta correlación."
```

```
)
```

```
print(
```

```
    "\n3. La reducción dimensional disminuye el tráfico "
```

```
    "de datos enviado al PLC, reduce el consumo de ancho "
```

```
    "de banda y mejora el procesamiento en tiempo real."
```

```
)
```

```
In [5]: %runfile 'C:/Users/liclo/Documents/Sin título3.py' --wdir
```

Primeras filas del dataset:

	temp_nucleo	temp_escape	presion_interna	...	corriente	ruido_db	eficiencia
0	102.483571	102.841358	34.055723	...	15.806139	64.362972	89.751643
1	99.308678	99.869463	44.006250	...	15.657457	57.595249	90.069132
2	103.238443	104.321494	50.052437	...	15.819982	63.975226	89.676156
3	107.615149	108.668951	50.469806	...	15.371064	68.108175	89.238485
4	98.829233	97.451564	45.499345	...	15.037717	60.810906	90.117077

[5 rows x 12 columns]

Resumen de Varianza:

	Componente	Varianza Explicada (%)	Varianza Acumulada (%)
0	1	43.17	43.17
1	2	10.95	54.12
2	3	9.54	63.65
3	4	8.73	72.39
4	5	7.66	80.05
5	6	7.07	87.12
6	7	6.35	93.47
7	8	4.66	98.13
8	9	1.46	99.59
9	10	0.23	99.81
10	11	0.19	100.00
11	12	0.00	100.00

Componentes necesarios para explicar al menos 85% de la varianza:
6

Loadings de PC1 y PC2:

	PC1	PC2
temp_nucleo	-0.427	-0.026
temp_escape	-0.419	-0.044
presion_interna	0.050	-0.210
vibracion_motor	-0.388	0.110
flujo_gas	0.001	0.542
oxigeno	0.047	0.437
co2	-0.395	-0.063
humedad	0.000	0.435
voltaje	0.010	0.496
corriente	0.070	0.060
ruido_db	-0.379	0.113
eficiencia	0.427	0.026


```
Variables más importantes en PC1:
eficiencia      0.426724
temp_nucleo     0.426724
temp_escape     0.418783
co2             0.395314
vibracion_motor 0.388235
Name: PC1, dtype: float64
```

```
Variables más importantes en PC2:
flujo_gas       0.541607
voltaje         0.496269
oxigeno         0.437444
humedad         0.435237
presion_interna 0.209541
Name: PC2, dtype: float64
```

Matriz de correlación:

	temp_nucleo	temp_escape	...	ruido_db	eficiencia
temp_nucleo	1.00	0.98	...	0.74	-1.00
temp_escape	0.98	1.00	...	0.71	-0.98
presion_interna	-0.13	-0.14	...	-0.02	0.13
vibracion_motor	0.77	0.74	...	0.97	-0.77
flujo_gas	-0.03	-0.06	...	0.09	0.03
oxigeno	-0.09	-0.10	...	-0.05	0.09
co2	0.88	0.86	...	0.68	-0.88
humedad	-0.00	-0.03	...	0.03	0.00
voltaje	-0.03	-0.02	...	0.01	0.03
corriente	-0.13	-0.10	...	-0.13	0.13
ruido_db	0.74	0.71	...	1.00	-0.74
eficiencia	-1.00	-0.98	...	-0.74	1.00

[12 rows x 12 columns]

Posibles sensores redundantes ($| \text{correlación} | > 0.85$):

```
temp_nucleo <--> temp_escape (corr=0.979)
temp_nucleo <--> co2 (corr=0.880)
temp_nucleo <--> eficiencia (corr=-1.000)
temp_escape <--> co2 (corr=0.861)
temp_escape <--> eficiencia (corr=-0.979)
vibracion_motor <--> ruido_db (corr=0.973)
co2 <--> eficiencia (corr=-0.880)
```

Práctica 5: Reducción de Dimensiones en Tráfico de Red (Cyber-Systems)

Unidad 2: Tratamiento de Datos – TecNM

Contexto Profesional

Eres el líder de seguridad en un Centro de Datos. Tienes sensores que capturan 10 métricas distintas por cada conexión. Analizar 10 dimensiones en tiempo real es costoso. Tu objetivo es encontrar los "Componentes Principales" que describan el estado de la red (Normal vs. Anomalía) reduciendo la carga computacional.

Paso 1: Generación y Exploración de Datos

Primero, simulamos los datos de red. Observa que algunas variables están muy relacionadas (ej. bytes y paquetes).

```
set.seed(123)
n <- 300
# Datos simulados de tráfico
datos_red <- data.frame(
  duracion_ms = rnorm(n, 50, 10),
  paquetes_enviados = rnorm(n, 100, 20),
  bytes_enviados = NA, # Se calculará con correlación
  errores_checksum = rpois(n, 2),
  reintentos_tcp = NA,
  latencia_avg = rnorm(n, 15, 5),
  jitter = rnorm(n, 2, 0.5),
  carga_cpu_router = NA,
  uso_memoria_sw = rnorm(n, 40, 10),
  peticiones_http = rnorm(n, 200, 50)
)

# Creamos dependencias (Redundancia)
datos_red$bytes_enviados <- datos_red$paquetes_enviados * 1500 + rnorm(n,
0, 500)
datos_red$reintentos_tcp <- datos_red$errores_checksum * 1.5 + rnorm(n,
0, 0.5)
datos_red$carga_cpu_router <- (datos_red$paquetes_enviados * 0.4) +
(datos_red$latencia_avg * 0.2)

head(datos_red)
```

Interpretación Docente: Aquí el alumno debe notar que `bytes_enviados` es casi un espejo de `paquetes_enviados`. En ingeniería, esto es **redundancia**. El PCA detectará que no necesitamos ambas columnas para entender el flujo.

Paso 2: Estandarización (Scaling)

El PCA es sensible a las magnitudes. Si no escalamos, la variable `bytes_enviados` (que está en miles) dominará el análisis sobre `jitter` (que está en unidades).

```
# Ejecutamos PCA con scale. = TRUE
pca_red <- prcomp(datos_red, center = TRUE, scale. = TRUE)
```

Interpretación Docente: Escalar transforma los datos para que todos tengan media 0 y desviación 1. Esto garantiza que cada métrica de red tenga el mismo "voto" inicial en el algoritmo.

Paso 3: Análisis de Varianza (Varianza Explicada)

¿Cuánta información logramos retener?

```
summary(pca_red)
plot(pca_red, type = "l", main = "Scree Plot: Tráfico de Red")
```

Interpretación Docente: El alumno debe mirar la fila "Cumulative Proportion".

- **Punto clave:** Si el PC1 y PC2 suman > 0.70 (70%), significa que con solo 2 nuevas variables podemos explicar la mayor parte de lo que pasa en la red. El "codo" en la gráfica nos dice gráficamente dónde parar.

Paso 4: Interpretación de Cargas (Loadings)

¿Qué significan los nuevos componentes?

```
# Ver pesos de las variables originales en los componentes
round(pca_red$rotation[, 1:2], 3)
```

Interpretación Docente: > * Si el PC1 tiene valores altos en `paquetes`, `bytes` y `carga_cpu`, el alumno interpreta que **PC1 = Volumen de Tráfico**.

- Si el PC2 tiene valores altos en `latencia` y `jitter`, el alumno interpreta que **PC2 = Calidad/Estabilidad de Conexión**.
- ¡Hemos convertido 10 variables en 2 conceptos de ingeniería claros!

Paso 5: Visualización (Biplot)

El diagnóstico final.

```
biplot(pca_red, cex = 0.8, main = "Mapa de Estado de Red")
```

Interpretación Docente: > * **Flechas paralelas:** Indican variables redundantes (ej. `bytes` y `paquetes`).

- **Flechas largas:** Son las variables que más influyen en el sistema.
- **Puntos aislados:** Son conexiones anómalas (posibles ataques o fallas) que se alejan del comportamiento normal de la red.

```
# =====
```

```
# PCA PARA ANÁLISIS DE TRÁFICO DE RED
```

```
# Centro de Datos - Detección de Anomalías
```

```
# =====
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA
```

```
# -----
```

```
# PASO 1: Generación de Datos Simulados
```

```
# -----
```

```
np.random.seed(123)
```

```
n = 300
```

```
datos_red = pd.DataFrame({
```

```
    'duracion_ms': np.random.normal(50, 10, n),
```

```
'paquetes_enviados': np.random.normal(100, 20, n),  
'errores_checksum': np.random.poisson(2, n),  
'latencia_avg': np.random.normal(15, 5, n),  
'jitter': np.random.normal(2, 0.5, n),  
'uso_memoria_sw': np.random.normal(40, 10, n),  
'peticiones_http': np.random.normal(200, 50, n)  
})
```

```
# Crear dependencias (redundancia)
```

```
datos_red['bytes_enviados'] = (  
    datos_red['paquetes_enviados'] * 1500 +  
    np.random.normal(0, 500, n)  
)
```

```
datos_red['reintentos_tcp'] = (  
    datos_red['errores_checksum'] * 1.5 +  
    np.random.normal(0, 0.5, n)  
)
```

```
datos_red['carga_cpu_router'] = (  
    datos_red['paquetes_enviados'] * 0.4 +  
    datos_red['latencia_avg'] * 0.2  
)
```

```
# Reordenar columnas

datos_red = datos_red[

    [

        'duracion_ms',

        'paquetes_enviados',

        'bytes_enviados',

        'errores_checksum',

        'reintentos_tcp',

        'latencia_avg',

        'jitter',

        'carga_cpu_router',

        'uso_memoria_sw',

        'peticiones_http'

    ]

]

print("\nPrimeras filas del dataset:")

print(datos_red.head())

# -----

# Correlación entre variables

# -----
```

```
print("\nMatriz de correlación:")

print(datos_red.corr().round(2))


# -----

# PASO 2: Estandarización

# -----


scaler = StandardScaler()

datos_escalados = scaler.fit_transform(datos_red)


print("\nDatos estandarizados correctamente.")


# -----

# PASO 3: Aplicación de PCA

# -----


pca = PCA()

componentes = pca.fit_transform(datos_escalados)


# Varianza explicada

varianza = pca.explained_variance_ratio_

varianza_acumulada = np.cumsum(varianza)
```

```
print("\nVarianza explicada por componente:")
```

```
for i, v in enumerate(varianza):
```

```
    print(f"PC {i+1}: {v:.4f}")
```

```
print("\nVarianza acumulada:")
```

```
for i, v in enumerate(varianza_acumulada):
```

```
    print(f"PC {i+1}: {v:.4f}")
```

```
# -----
```

```
# Scree Plot
```

```
# -----
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(
```

```
    range(1, len(varianza)+1),
```

```
    varianza,
```

```
    marker='o'
```

```
)
```

```
plt.title("Scree Plot - Tráfico de Red")
```

```
plt.xlabel("Componentes Principales")
```

```
plt.ylabel("Varianza Explicada")
```

```
plt.grid(True)
```



```
plt.show()
```

```
# -----
```

```
# PASO 4: Cargas (Loadings)
```

```
# -----
```

```
loadings = pd.DataFrame(  
    pca.components_.T,  
    columns=[f'PC {i+1}' for i in range(len(datos_red.columns))],  
    index=datos_red.columns  
)
```

```
print("\nCargas de PC1 y PC2:")
```

```
print(loadings[['PC1', 'PC2']].round(3))
```

```
# -----
```

```
# Interpretación automática
```

```
# -----
```

```
print("\nVariables más importantes en PC1:")
```

```
print(loadings['PC1'].abs().sort_values(ascending=False).head())
```

```
print("\nVariables más importantes en PC2:")
```

```
print(loadings['PC2'].abs().sort_values(ascending=False).head())
```

```
# -----
```

```
# PASO 5: Biplot
```

```
# -----
```

```
plt.figure(figsize=(10,8))
```

```
# Observaciones
```

```
plt.scatter(  
    componentes[:, 0],  
    componentes[:, 1],  
    alpha=0.6  
)
```

```
# Flechas de variables
```

```
for i, variable in enumerate(datos_red.columns):
```

```
    plt.arrow(  
        0,  
        0,  
        loadings.iloc[i, 0] * 5,  
        loadings.iloc[i, 1] * 5,  
        alpha=0.7
```

)

```
plt.text(  
    loadings.iloc[i, 0] * 5.2,  
    loadings.iloc[i, 1] * 5.2,  
    variable,  
    fontsize=9  
)
```

```
plt.axhline(0)
```

```
plt.axvline(0)
```

```
plt.xlabel(  
    f"PC1 ( {varianza[0]*100:.2f}% varianza)"  
)
```

```
plt.ylabel(  
    f"PC2 ( {varianza[1]*100:.2f}% varianza)"  
)
```

```
plt.title("Mapa de Estado de Red (Biplot PCA)")
```

```
plt.grid(True)
```

```
plt.show()
```

```

# -----

# Detección simple de anomalías

# -----


distancias = np.sqrt(
    componentes[:,0]**2 +
    componentes[:,1]**2
)

umbral = np.percentile(distancias, 95)

anomalias = np.where(distancias > umbral)[0]

print(f"\nNúmero de conexiones anómalas detectadas: {len(anomalias)}")
print("Índices de anomalías:")
print(anomalias)

# Visualización de anomalías

plt.figure(figsize=(10,7))

plt.scatter(
    componentes[:,0],

```

```
componentes[:,1],  
alpha=0.5,  
label='Normal'  
)
```

```
plt.scatter(  
    componentes[anomalias,0],  
    componentes[anomalias,1],  
    s=80,  
    marker='x',  
    label='Anomalía'  
)
```

```
plt.xlabel("PC1")  
plt.ylabel("PC2")  
plt.title("Detección de Anomalías usando PCA")  
plt.legend()  
plt.grid(True)  
plt.show()
```

```
In [6]: %runfile 'C:/Users/liclo/Documents/Sin título4.py' --wdir
```

Primeras filas del dataset:

	duracion_ms	paquetes_enviados	...	uso_memoria_sw	peticiones_http
0	39.143694	115.301097	...	56.215306	169.596990
1	59.973454	83.420223	...	51.652583	131.539070
2	52.829785	86.816974	...	25.428999	234.862122
3	34.937053	112.222471	...	53.834913	159.001548
4	44.213997	97.119733	...	39.616217	257.375376

[5 rows x 10 columns]

Matriz de correlación:

	duracion_ms	...	peticiones_http
duracion_ms	1.00	...	0.11
paquetes_enviados	-0.09	...	0.08
bytes_enviados	-0.09	...	0.08
errores_checksum	-0.03	...	0.06
reintentos_tcp	-0.02	...	0.05
latencia_avg	-0.07	...	-0.05
jitter	0.09	...	-0.07
carga_cpu_router	-0.10	...	0.07
uso_memoria_sw	-0.02	...	-0.05
peticiones_http	0.11	...	1.00

[10 rows x 10 columns]

Datos estandarizados correctamente.

Varianza explicada por componente:

PC1: 0.3028
PC2: 0.1981
PC3: 0.1171
PC4: 0.1060
PC5: 0.0990
PC6: 0.0938
PC7: 0.0807
PC8: 0.0024
PC9: 0.0000
PC10: 0.0000

```
Varianza acumulada:
PC1: 0.3028
PC2: 0.5009
PC3: 0.6180
PC4: 0.7240
PC5: 0.8230
PC6: 0.9168
PC7: 0.9975
PC8: 1.0000
PC9: 1.0000
PC10: 1.0000
```

Cargas de PC1 y PC2:

	PC1	PC2
duracion_ms	-0.081	-0.017
paquetes_enviados	0.571	-0.056
bytes_enviados	0.571	-0.055
errores_checksum	0.062	0.699
reintentos_tcp	0.061	0.699
latencia_avg	0.052	0.015
jitter	-0.027	-0.081
carga_cpu_router	0.572	-0.054
uso_memoria_sw	-0.019	0.034
peticiones_http	0.064	0.067

Variables más importantes en PC1:

carga_cpu_router	0.571559
paquetes_enviados	0.570702
bytes_enviados	0.570656
duracion_ms	0.080600
peticiones_http	0.063653

Name: PC1, dtype: float64

Variables más importantes en PC2:

errores_checksum	0.699348
reintentos_tcp	0.699342
jitter	0.081124
peticiones_http	0.067082
paquetes_enviados	0.056049

Name: PC2, dtype: float64

Número de conexiones anómalas detectadas: 15

Índices de anomalías:

```
[ 33  58  70 119 137 178 182 188 208 212 246 264 290 293 299]
```

title: "Práctica 6: Optimización de Sensores en Smart Cities" subtitle: "Unidad 2: Tratamiento de Datos - TecNM" author: "Guía Paso a Paso con Interpretación" output: html_document: toc: true theme: united

```
knitr::opts_chunk.set(echo = TRUE)
library(tidyverse)
```

Contexto Profesional

Imagina que trabajas para el departamento de Innovación Tecnológica de una metrópoli. Han instalado 10 sensores en cada intersección principal para medir: calidad del aire, ruido, tráfico y clima. Transmitir las 10 señales por cada nodo de la ciudad satura la red de bajo consumo (LoRaWAN). Tu misión es aplicar **PCA** para reducir estas métricas a los componentes esenciales que describan el "Estado de la Ciudad".

Paso 1: Generación de Datos Urbanos

Simulamos el comportamiento de los sensores. Nota cómo variables como el CO2 y la densidad de vehículos suelen moverse juntas.

```
set.seed(456)
n <- 400
# Datos base de sensores urbanos
datos_urbanos <- data.frame(
  temp_ambiente = rnorm(n, 28, 4),
  humedad_rel = rnorm(n, 60, 10),
  co2_ppm = NA,           # Relacionado con tráfico
  no2_ppb = NA,           # Relacionado con tráfico
  particulas_pm25 = NA,   # Relacionado con CO2
  nivel_ruido_db = NA,    # Relacionado con vehículos
  densidad_vehiculos = rnorm(n, 120, 30),
  velocidad_viento = rnorm(n, 15, 5),
  radiacion_solar = rnorm(n, 800, 100),
  conteo_peatones = rnorm(n, 50, 15)
)

# Creación de Redundancia y Correlación (Lógica de ciudad)
datos_urbanos$co2_ppm <- (datos_urbanos$densidad_vehiculos * 3.5) +
  rnorm(n, 300, 50)
datos_urbanos$no2_ppb <- (datos_urbanos$co2_ppm * 0.1) + rnorm(n, 5, 2)
datos_urbanos$particulas_pm25 <- (datos_urbanos$co2_ppm * 0.05) +
  rnorm(n, 10, 3)
datos_urbanos$nivel_ruido_db <- (datos_urbanos$densidad_vehiculos * 0.2)
+ 50 + rnorm(n, 0, 5)

head(datos_urbanos)
```


Interpretación Docente: El estudiante debe observar que el CO₂, el NO₂ y las partículas PM_{2.5} están fuertemente ligados a la densidad_vehiculos. En lugar de monitorear 4 sensores que dicen casi lo mismo, el PCA nos dirá que podemos crear un solo "Índice de Contaminación Vehicular".

Paso 2: Estandarización de las Métricas

¿Por qué es obligatorio el comando `scale. = TRUE`? Porque el CO₂ se mide en cientos de unidades, mientras que la humedad en porcentajes.

```
# Ejecución del PCA con escalamiento centrado
pca_ciudad <- prcomp(datos_urbanos, center = TRUE, scale. = TRUE)
```

Interpretación Docente: Sin estandarización, las variables con rangos numéricos más grandes "secuestrarían" el análisis. El escalamiento permite que el sensor de temperatura tenga el mismo peso inicial que el de CO₂ para detectar cambios en el patrón de la ciudad.

Paso 3: Análisis de Varianza Acumulada

Determinamos cuánta información "sobrevive" a la reducción.

```
# Resumen de varianza por componente
summary(pca_ciudad)

# Visualización del Scree Plot
plot(pca_ciudad, type = "l",
     main = "Scree Plot: Sensores Smart City",
     ylab = "Varianza Explicada", xlab = "Componentes")
abline(h = 1, col="red", lty=2)
```

Interpretación Docente: > * **Varianza Proporcional:** Si el PC1 explica el 55% de la varianza, significa que más de la mitad de la complejidad de la ciudad se resume en una sola dimensión.

- **Criterio de Selección:** Los alumnos deben buscar el punto donde la curva se aplana. Si los primeros 3 componentes explican el 85%, el ahorro de transmisión de datos para la ciudad es del **70%**.

Paso 4: Interpretación de los Componentes (Loadings)

Traducción de matemáticas a ingeniería urbana.

```
# Pesos de las variables originales
round(pca_ciudad$rotation[, 1:2], 3)
```

Interpretación Docente: El alumno debe realizar la siguiente deducción:

- **PC1:** Si tiene cargas altas en `vehiculos`, `co2`, `no2` y `ruido`, el **PC1 = Actividad e Impacto Vehicular**.
- **PC2:** Si tiene cargas altas en `temp`, `humedad` y `radiacion`, el **PC2 = Factor Climático/Meteorológico**.
- Hemos simplificado el tablero de control de la ciudad de 10 indicadores a solo 2 macro-indicadores.

Paso 5: El Biplot de Diagnóstico Urbano

Visualizamos el estado de la metrópoli.

```
biplot(pca_ciudad, cex = 0.7, main = "Mapa Biplot de Estado Urbano")
```

Interpretación Docente: > * **Flechas paralelas:** Indican sensores que se pueden consolidar. Por ejemplo, si las flechas de `paquetes_pm25` y `no2_ppb` apuntan al mismo lugar, son redundantes.

- **Ángulo de 90 grados:** Si la flecha de `viento` es perpendicular a la de `vehiculos`, el viento no depende del tráfico.
- **Puntos atípicos:** Aquellos puntos muy alejados en la dirección de la flecha de `ruido` y `co2` representan horas pico o incidentes de tráfico severo detectados por el sistema.

```
# =====
```

```
# PCA PARA SENSORES DE SMART CITY
```

```
# Autor: ChatGPT
```

```
# =====
```

```
# Librerías
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA
```

```
# =====  
  
# PASO 1: GENERACIÓN DE DATOS URBANOS  
  
# =====  
  
  
np.random.seed(456)  
  
  
n = 400  
  
  
# Datos base de sensores urbanos  
  
datos_urbanos = pd.DataFrame({  
    'temp_ambiente': np.random.normal(28, 4, n),  
    'humedad_rel': np.random.normal(60, 10, n),  
    'co2_ppm': np.nan,  
    'no2_ppb': np.nan,  
    'particulas_pm25': np.nan,  
    'nivel_ruido_db': np.nan,  
    'densidad_vehiculos': np.random.normal(120, 30, n),  
    'velocidad_viento': np.random.normal(15, 5, n),  
    'radiacion_solar': np.random.normal(800, 100, n),  
    'conteo_peatones': np.random.normal(50, 15, n)  
})
```

```
# Creación de correlaciones urbanas
```

```
datos_urbanos['co2_ppm'] = (  
    datos_urbanos['densidad_vehiculos'] * 3.5  
    + np.random.normal(300, 50, n)  
)
```

```
datos_urbanos['no2_ppb'] = (  
    datos_urbanos['co2_ppm'] * 0.1  
    + np.random.normal(5, 2, n)  
)
```

```
datos_urbanos['particulas_pm25'] = (  
    datos_urbanos['co2_ppm'] * 0.05  
    + np.random.normal(10, 3, n)  
)
```

```
datos_urbanos['nivel_ruido_db'] = (  
    datos_urbanos['densidad_vehiculos'] * 0.2  
    + 50  
    + np.random.normal(0, 5, n)  
)
```

```
print("\nPrimeras filas del dataset:")
```

```
print(datos_urbanos.head())
```

```
# =====
```

```
# PASO 2: ESTANDARIZACIÓN DE DATOS
```

```
# =====
```

```
scaler = StandardScaler()
```

```
datos_escalados = scaler.fit_transform(datos_urbanos)
```

```
# =====
```

```
# PASO 3: EJECUCIÓN DEL PCA
```

```
# =====
```

```
pca = PCA()
```

```
componentes = pca.fit_transform(datos_escalados)
```

```
# Varianza explicada
```

```
varianza = pca.explained_variance_ratio_
```

```
varianza_acumulada = np.cumsum(varianza)
```

```
# Tabla resumen
```

```

resumen = pd.DataFrame({
    "Componente": np.arange(1, len(varianza)+1),
    "Varianza Explicada (%)": np.round(varianza * 100, 2),
    "Varianza Acumulada (%)": np.round(varianza_acumulada * 100, 2)
})

print("\nResumen PCA:")
print(resumen)

# =====

# PASO 4: SCREE PLOT

# =====

plt.figure(figsize=(8,5))
plt.plot(
    range(1, len(varianza)+1),
    varianza * 100,
    marker='o'
)

plt.title("Scree Plot: Sensores Smart City")
plt.xlabel("Componentes Principales")
plt.ylabel("Varianza Explicada (%)")

```

```
plt.grid(True)
```

```
plt.show()
```

```
# =====
```

```
# PASO 5: ANÁLISIS DE VARIANZA ACUMULADA
```

```
# =====
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(
```

```
    range(1, len(varianza_acumulada)+1),
```

```
    varianza_acumulada * 100,
```

```
    marker='s'
```

```
)
```

```
plt.axhline(
```

```
    y=85,
```

```
    linestyle='--',
```

```
    label='85% Varianza'
```

```
)
```

```
plt.title("Varianza Acumulada")
```

```
plt.xlabel("Número de Componentes")
```

```
plt.ylabel("Varianza Acumulada (%)")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
# Número mínimo de componentes para explicar 85%
```

```
componentes_85 = np.argmax(varianza_acumulada >= 0.85) + 1
```

```
print(
```

```
    f"\nComponentes necesarios para explicar al menos "
```

```
    f"el 85% de la varianza: {componentes_85}"
```

```
)
```

```
# =====
```

```
# PASO 6: LOADINGS (PESOS DE VARIABLES)
```

```
# =====
```

```
loadings = pd.DataFrame(
```

```
    pca.components_.T,
```

```
    columns=[f"PC {i+1}" for i in range(len(datos_urbanos.columns))],
```

```
    index=datos_urbanos.columns
```

```
)
```

```
print("\nLoadings de PC1 y PC2:")
```



```
print(loadings[['PC1', 'PC2']].round(3))
```

```
# =====
```

```
# PASO 7: INTERPRETACIÓN AUTOMÁTICA
```

```
# =====
```

```
print("\nVariables con mayor peso en PC1:")
```

```
print(loadings["PC1"].abs().sort_values(ascending=False).head())
```

```
print("\nVariables con mayor peso en PC2:")
```

```
print(loadings["PC2"].abs().sort_values(ascending=False).head())
```

```
# =====
```

```
# PASO 8: BIPLLOT
```

```
# =====
```

```
plt.figure(figsize=(10,8))
```

```
# Puntos (observaciones)
```

```
plt.scatter(
```

```
    componentes[:,0],
```

```
    componentes[:,1],
```

```
    alpha=0.5
```

```
)
```

```
# Escalado visual de flechas
```

```
escala = 4
```

```
for i, variable in enumerate(datos_urbanos.columns):
```

```
    plt.arrow(
```

```
        0,
```

```
        0,
```

```
        loadings.iloc[i,0] * escala,
```

```
        loadings.iloc[i,1] * escala,
```

```
        head_width=0.05
```

```
    )
```

```
    plt.text(
```

```
        loadings.iloc[i,0] * escala * 1.15,
```

```
        loadings.iloc[i,1] * escala * 1.15,
```

```
        variable,
```

```
        fontsize=9
```

```
    )
```

```
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")
```

```
plt.title("Biplot: Estado Urbano")
```

```
plt.grid(True)
```

```
plt.show()
```

```
# =====
```

```
# PASO 9: REDUCCIÓN DE DIMENSIONALIDAD
```

```
# =====
```

```
pca_reducido = PCA(n_components=2)
```

```
datos_reducidos = pca_reducido.fit_transform(datos_escalados)
```

```
df_reducido = pd.DataFrame(
```

```
    datos_reducidos,
```

```
    columns=["Actividad_Vehicular", "Factor_Climatico"]
```

```
)
```

```
print("\nDatos reducidos a dos macro-indicadores:")
```

```
print(df_reducido.head())
```

```
# =====
```

```
# CONCLUSIONES AUTOMÁTICAS
```

```
# =====
```

```
ahorro = (1 - 2/10) * 100
```

```
print("\n===== RESULTADOS =====")
```

```
print(f"Sensores originales: 10")
```

```
print(f"Componentes finales: 2")
```

```
print(f"Reducción de dimensiones: {ahorro:.0f}%")
```

```
print(
```

```
    f"Varianza explicada por los dos primeros componentes: "
```

```
    f"{pca_reducido.explained_variance_ratio_.sum()*100:.2f}%"
```

```
)
```

```
print("=====")
```

```
In [7]: %runfile 'C:/Users/Liclo/Documents/Sin título5.py' --wdir
```

```
Primeras filas del dataset:
```

	temp_ambiente	humedad_rel	...	radiacion_solar	conteo_peatones
0	25.327486	45.261746	...	914.157550	57.722465
1	26.007162	67.870763	...	965.173743	38.786281
2	30.474303	63.935109	...	834.292432	52.585696
3	30.274769	50.079965	...	796.567047	42.961262
4	33.402038	68.525178	...	678.768816	19.779909

```
[5 rows x 10 columns]
```

Resumen PCA:

	Componente	Varianza Explicada (%)	Varianza Acumulada (%)
0	1	41.80	41.80
1	2	10.91	52.71
2	3	10.23	62.94
3	4	9.90	72.83
4	5	9.77	82.60
5	6	9.13	91.73
6	7	5.19	96.92
7	8	1.79	98.71
8	9	1.14	99.86
9	10	0.14	100.00

Componentes necesarios para explicar al menos el 85% de la varianza: 6

Loadings de PC1 y PC2:

	PC1	PC2
temp_ambiente	0.016	-0.482
humedad_rel	-0.010	0.630
co2_ppm	0.475	0.017
no2_ppb	0.469	0.012
particulas_pm25	0.441	0.052
nivel_ruido_db	0.374	-0.078
densidad_vehiculos	0.461	-0.038
velocidad_viento	0.038	-0.287
radiacion_solar	-0.065	-0.339
conteo_peatones	-0.032	-0.402

Variables con mayor peso en PC1:

```
co2_ppm          0.475123
no2_ppb          0.469194
densidad_vehiculos 0.460876
particulas_pm25   0.441467
nivel_ruido_db    0.373818
Name: PC1, dtype: float64
```

Variables con mayor peso en PC2:

```
humedad_rel      0.630481
temp_ambiente    0.482418
conteo_peatones  0.401702
radiacion_solar  0.339210
velocidad_viento 0.287438
Name: PC2, dtype: float64
```

Datos reducidos a dos macro-indicadores:

	Actividad_Vehicular	Factor_Climatico
0	0.697176	-1.291000
1	-0.982442	0.578131
2	0.069760	0.192515
3	-2.569506	-0.569034
4	-1.265095	1.123553

===== RESULTADOS =====

Sensores originales: 10

Componentes finales: 2

Reducción de dimensiones: 80%

Varianza explicada por los dos primeros componentes: 52.71%

=====

Práctica 7: Implementación de Modelos de Machine Learning

Unidad 3: Modelado - Ciencia de Datos (TecNM)

```
knitr::opts_chunk.set(echo = TRUE)
library(tidyverse)
library(caret) # Librería estándar para entrenamiento de modelos
library(glmnet) # Para modelos Lasso
```

Contexto Profesional

La empresa "DataSystems S.A." desea automatizar su departamento de Recursos Humanos. Necesitan predecir:

1. **Regresión:** El salario esperado de un nuevo perfil tecnológico.
2. **Clasificación:** Si un candidato permanecerá en la empresa más de 2 años (Retención).
3. **Clustering:** Agrupar empleados actuales para planes de capacitación personalizados.

Paso 1: Preparación del Dataset

Generamos datos que incluyan experiencia, nivel de certificaciones, habilidades sociales y salario.

```
set.seed(789)
n <- 500
df_empleados <- data.frame(
  experiencia = rnorm(n, 5, 2),
  certificaciones = rpois(n, 3),
  habilidades_sociales = runif(n, 1, 10),
  remoto = sample(c(0, 1), n, replace = TRUE)
)

# Variable objetivo continua (Salario)
df_empleados$salario <- 25000 + (df_empleados$experiencia * 5000) +
  (df_empleados$certificaciones * 2000) + rnorm(n,
0, 3000)

# Variable objetivo binaria (Retención: 1=Se queda, 0=Se va)
# Probabilidad depende de salario y habilidades sociales
prob <- plogis(-2 + 0.0001*df_empleados$salario +
0.2*df_empleados$habilidades_sociales)
df_empleados$retencion <- factor(ifelse(runif(n) < prob, 1, 0))

head(df_empleados)
```

Interpretación Docente: El alumno debe notar que estamos creando un ecosistema de datos donde las variables tienen relaciones lógicas que el modelo debe "descubrir".

Paso 2: Regresión Lineal (Predecir Salario)

Dividimos los datos en entrenamiento (80%) y prueba (20%).

```
set.seed(123)
indice <- createDataPartition(df_empleados$salario, p = 0.8, list =
FALSE)
train_data <- df_empleados[indice, ]
test_data <- df_empleados[-indice, ]

# Entrenamiento del modelo
modelo_salario <- lm(salario ~ experiencia + certificaciones, data =
train_data)
summary(modelo_salario)
```

Interpretación Docente: Aquí el alumno analiza los coeficientes. ¿Cuánto aumenta el salario por cada año de experiencia extra? (\$5000 aprox. en este caso).

Paso 3: Clasificación KNN (Predecir Retención)

Usaremos KNN para clasificar si un empleado se queda o se va. ¡Recordar que KNN requiere escalamiento!

```
# Caret escala automáticamente con preProcess = c("center", "scale")
modelo_knn <- train(retencion ~ experiencia + habilidades_sociales +
salario,
                    data = train_data,
                    method = "knn",
                    preProcess = c("center", "scale"),
                    tuneLength = 5)

print(modelo_knn)
```

Interpretación Docente: El estudiante debe observar cómo cambia la precisión (Accuracy) según el valor de k (número de vecinos).

Paso 4: Clustering No Supervisado (K-Means)

Agruparemos a los empleados basándonos solo en su experiencia y salario.

```
datos_clustering <- df_empleados %>% select(experiencia, salario) %>%
scale()
set.seed(456)
clusters <- kmeans(datos_clustering, centers = 3)

# Visualización
df_empleados$cluster <- as.factor(clusters$cluster)
```



```
ggplot(df_empleados, aes(x = experiencia, y = salario, color = cluster))
+
  geom_point() +
  labs(title = "Segmentación de Empleados por Perfil") +
  theme_minimal()
```

Interpretación Docente: El alumno identifica 3 grupos: Junior, Mid y Senior. Esto demuestra cómo K-Means encuentra estructuras sin que le digamos qué buscar.

```
# =====
```

```
# PROYECTO: DataSystems S.A.
```

```
# Regresión, Clasificación y Clustering de Empleados
```

```
# =====
```

```
# Librerías
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.metrics import (
```

```

    mean_squared_error,

    r2_score,

    accuracy_score,

    confusion_matrix,

    classification_report
)

# =====

# PASO 1: GENERACIÓN DEL DATASET

# =====


np.random.seed(789)


n = 500


df_empleados = pd.DataFrame({

    'experiencia': np.random.normal(5, 2, n),

    'certificaciones': np.random.poisson(3, n),

    'habilidades_sociales': np.random.uniform(1, 10, n),

    'remoto': np.random.choice([0, 1], n)

})


# Variable objetivo continua (Salario)

```

```

df_empleados['salario'] = (
    25000
    + df_empleados['experiencia'] * 5000
    + df_empleados['certificaciones'] * 2000
    + np.random.normal(0, 3000, n)
)

# Variable objetivo binaria (Retención)
prob = 1 / (
    1 + np.exp(
        -(-2
            + 0.0001 * df_empleados['salario']
            + 0.2 * df_empleados['habilidades_sociales'])
    )
)

df_empleados['retencion'] = np.where(
    np.random.rand(n) < prob,
    1,
    0
)

print("\nPrimeras filas del dataset:")

```

```
print(df_empleados.head())
```

```
# =====
```

```
# PASO 2: REGRESIÓN LINEAL
```

```
# PREDICCIÓN DEL SALARIO
```

```
# =====
```

```
X_reg = df_empleados[['experiencia', 'certificaciones']]
```

```
y_reg = df_empleados['salario']
```

```
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
```

```
    X_reg,
```

```
    y_reg,
```

```
    test_size=0.20,
```

```
    random_state=123
```

```
)
```

```
modelo_salario = LinearRegression()
```

```
modelo_salario.fit(X_train_reg, y_train_reg)
```

```
pred_salario = modelo_salario.predict(X_test_reg)
```

```

print("\n=====")

print("MODELO DE REGRESIÓN LINEAL")

print("=====")


print(f"Intercepto: {modelo_salario.intercept_:.2f}")


for variable, coef in zip(X_reg.columns, modelo_salario.coef_):

    print(f"{variable}: {coef:.2f}")


rmse = np.sqrt(mean_squared_error(y_test_reg, pred_salario))

r2 = r2_score(y_test_reg, pred_salario)


print(f"\nRMSE: {rmse:.2f}")

print(f"R²: {r2:.4f}")


# =====

# PASO 3: CLASIFICACIÓN KNN

# PREDICCIÓN DE RETENCIÓN

# =====


X_clf = df_empleados[

    ['experiencia', 'habilidades_sociales', 'salario']

]

```

```
y_clf = df_empleados['retencion']
```

```
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(  
    X_clf,  
    y_clf,  
    test_size=0.20,  
    random_state=123  
)
```

```
# Escalamiento + KNN
```

```
pipeline_knn = Pipeline([  
    ('scaler', StandardScaler()),  
    ('knn', KNeighborsClassifier(n_neighbors=5))  
])
```

```
pipeline_knn.fit(X_train_clf, y_train_clf)
```

```
pred_knn = pipeline_knn.predict(X_test_clf)
```

```
accuracy = accuracy_score(y_test_clf, pred_knn)
```

```
print("\n=====")
```

```

print("MODELO KNN - RETENCIÓN")

print("=====")

print(f'Accuracy: {accuracy:.4f}')

print("\nMatriz de Confusión:")

print(confusion_matrix(y_test_clf, pred_knn))

print("\nReporte de Clasificación:")

print(classification_report(y_test_clf, pred_knn))

# Comparación de distintos valores de K

print("\nAccuracy para distintos valores de K")

for k in range(1, 11):

    modelo_temp = Pipeline([

        ('scaler', StandardScaler()),

        ('knn', KNeighborsClassifier(n_neighbors=k))

    ])

    modelo_temp.fit(X_train_clf, y_train_clf)

```

```
pred_temp = modelo_temp.predict(X_test_clf)
```

```
acc = accuracy_score(y_test_clf, pred_temp)
```

```
print(f'K = {k}: {acc:.4f}')
```

```
# =====
```

```
# PASO 4: CLUSTERING K-MEANS
```

```
# =====
```

```
datos_cluster = df_empleados[  
    ['experiencia', 'salario']  
]
```

```
scaler = StandardScaler()
```

```
datos_cluster_escalados = scaler.fit_transform(datos_cluster)
```

```
kmeans = KMeans(  
    n_clusters=3,  
    random_state=456,  
    n_init=10
```



```
)
```

```
clusters = kmeans.fit_predict(datos_cluster_escalados)
```

```
df_empleados['cluster'] = clusters.astype(str)
```

```
print("\n=====")
```

```
print("TAMAÑO DE LOS CLUSTERS")
```

```
print("=====")
```

```
print(df_empleados['cluster'].value_counts())
```

```
# =====
```

```
# VISUALIZACIÓN
```

```
# =====
```

```
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(
```

```
    data=df_empleados,
```

```
    x='experiencia',
```

```
    y='salario',
```

```
    hue='cluster',
```

```

    palette='Set1'

)

plt.title('Segmentación de Empleados por Perfil')
plt.xlabel('Experiencia (años)')
plt.ylabel('Salario')
plt.grid(True)

plt.show()

# =====
# EJEMPLO DE PREDICCIÓN DE SALARIO
# =====

nuevo_empleado = pd.DataFrame({
    'experiencia': [7],
    'certificaciones': [4]
})

salario_estimado = modelo_salario.predict(nuevo_empleado)

print("\n=====")
print("PREDICCIÓN DE SALARIO")

```

```
print("=====")
```

```
print(f"Salario estimado: ${salario_estimado[0]:.2f}")
```

```
In [8]: %runfile 'C:/Users/liclo/Documents/Sin título6.py' --wdir
```

Primeras filas del dataset:

	experiencia	certificaciones	...	salario	retencion
0	2.783777	4	...	45873.792275	1
1	3.548563	1	...	43600.822391	1
2	6.045609	3	...	62247.070746	1
3	7.468884	5	...	72326.083333	1
4	5.193792	2	...	49629.856459	1

[5 rows x 6 columns]

=====

MODELO DE REGRESIÓN LINEAL

=====

Intercepto: 25575.49

experiencia: 4939.48

certificaciones: 2020.28

Intercepto: 25575.49

experiencia: 4939.48

certificaciones: 2020.28

RMSE: 3294.73

R²: 0.9184

=====

MODELO KNN - RETENCIÓN

=====

Accuracy: 0.9700

Matriz de Confusión:

```
[[ 0  3]
 [ 0 97]]
```

Reporte de Clasificación:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	3
1	0.97	1.00	0.98	97
accuracy			0.97	100
macro avg	0.48	0.50	0.49	100
weighted avg	0.94	0.97	0.96	100

```

Accuracy para distintos valores de K
K = 1: 0.9600
C:\Users\liclo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\liclo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\liclo\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

```

K = 2: 0.9100
K = 3: 0.9700
K = 4: 0.9700
K = 5: 0.9700
K = 6: 0.9700
K = 7: 0.9700
K = 8: 0.9700
K = 9: 0.9700
K = 10: 0.9700
C:\Users\liclo\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is
known to have a memory leak on Windows with MKL, when there are less chunks than available threads.
You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
    warnings.warn(

```

```

=====
TAMAÑO DE LOS CLUSTERS
=====
cluster
1    250
2    130
0    120
Name: count, dtype: int64

=====
PREDICCIÓN DE SALARIO
=====
Salario estimado: $68,232.97

```

Práctica 8: Clasificación de Intrusiones con KNN y Logística

Unidad 3: Modelado - TecNM

```
knitr::opts_chunk.set(echo = TRUE)
library(tidyverse)
library(caret)
```

Contexto Profesional

Un centro de datos del TecNM registra métricas de acceso. Tu tarea es clasificar si un acceso es **Seguro (0)** o **Ataque (1)** basándote en la latencia, el número de intentos fallidos y el tamaño del paquete.

Paso 1: Simulación de Tráfico de Red

```
set.seed(444)
n <- 400
df_red <- data.frame(
  latencia = rnorm(n, 20, 5),
  intentos_fallidos = rpois(n, 1),
  tamaño_paquete = rnorm(n, 500, 100)
)

# Lógica de clasificación: Ataque si intentos > 3 o latencia > 40
df_red$es_ataque <- factor(ifelse(df_red$intentos_fallidos > 2 |
df_red$latencia > 35, 1, 0))
```

Paso 2: Modelo de Regresión Logística

Entrenamos para obtener la probabilidad de que una conexión sea un ataque.

```
modelo_log <- glm(es_ataque ~ ., data = df_red, family = "binomial")
summary(modelo_log)
```

Interpretación Docente: El alumno debe observar el valor de z y el p-value. Si el p-value de `intentos_fallidos` es muy pequeño (< 0.05), significa que esta variable es un predictor crítico para detectar ataques.

Paso 3: Clasificación con KNN

Implementamos KNN para ver cómo los "vecinos" (conexiones similares) ayudan a clasificar.

```
control <- trainControl(method = "cv", number = 5)
```

```

modelo_knn <- train(es_ataque ~ ., data = df_red, method = "knn",
                    preProcess = c("center", "scale"),
                    trControl = control, tuneLength = 10)

plot(modelo_knn)

```

Interpretación Docente: El gráfico muestra la precisión (Accuracy) según el número de vecinos (\$k\$). El alumno debe identificar el \$k\$ **óptimo**. Un \$k\$ muy bajo puede sobreajustar (capturar ruido), mientras que uno muy alto puede suavizar demasiado el modelo y perder ataques reales.

Paso 4: Análisis de Resultados (Clustering K-Means)

Finalmente, agruparemos las conexiones sin usar la etiqueta de ataque para ver si el algoritmo "descubre" el comportamiento anómalo por sí solo.

```

datos_clus <- df_red %>% select(-es_ataque) %>% scale()
set.seed(111)
clusters <- kmeans(datos_clus, centers = 2)

```

```

# Comparación: ¿Qué tan bien coincidió el cluster con la realidad?
table(df_red$es_ataque, clusters$cluster)

```

Interpretación Docente: Si el cluster 1 contiene mayoritariamente "Ataques" y el cluster 2 "Seguros", el alumno concluye que los ataques tienen una firma digital tan distinta que son detectables incluso sin supervisión humana.

```

# =====

```

```

# PROYECTO: DETECCIÓN DE ATAQUES EN TRÁFICO DE RED

```

```

# Regresión Logística + KNN + K-Means

```

```

# =====

```

```

# Librerías

```

```

import numpy as np

```

```

import pandas as pd

```

```

import matplotlib.pyplot as plt

```

```

import seaborn as sns

```

```
from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.neighbors import KNeighborsClassifier

from sklearn.cluster import KMeans

from sklearn.metrics import (

    classification_report,

    confusion_matrix,

    accuracy_score

)
```

```
# =====
```

```
# PASO 1: SIMULACIÓN DE TRÁFICO DE RED
```

```
# =====
```

```
np.random.seed(444)
```

```
n = 400
```

```
df_red = pd.DataFrame({

    'latencia': np.random.normal(20, 5, n),

    'intentos_fallidos': np.random.poisson(1, n),

    'tamano_paquete': np.random.normal(500, 100, n)
```

```
}}
```

```
# Ataque si intentos > 2 o latencia > 35
```

```
df_red['es_ataque'] = np.where(  
    (df_red['intentos_fallidos'] > 2) |  
    (df_red['latencia'] > 35),  
    1,  
    0  
)
```

```
print("Primeras filas:")
```

```
print(df_red.head())
```

```
print("\nDistribución de clases:")
```

```
print(df_red['es_ataque'].value_counts())
```

```
# =====
```

```
# EXPLORACIÓN DE DATOS
```

```
# =====
```

```
print("\nEstadísticas descriptivas:")
```

```
print(df_red.describe())
```



```
sns.pairplot(  
    df_red,  
    hue='es_ataque',  
    diag_kind='hist'  
)  
plt.show()
```

```
# =====  
# PREPARACIÓN DE DATOS  
# =====
```

```
X = df_red.drop('es_ataque', axis=1)  
y = df_red['es_ataque']
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.30,  
    random_state=42,  
    stratify=y  
)
```

```
# Escalamiento
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# =====
```

```
# PASO 2: REGRESIÓN LOGÍSTICA
```

```
# =====
```

```
modelo_log = LogisticRegression(max_iter=1000)
```

```
modelo_log.fit(X_train_scaled, y_train)
```

```
pred_log = modelo_log.predict(X_test_scaled)
```

```
print("\n=====")
```

```
print("REGRESIÓN LOGÍSTICA")
```

```
print("=====")
```

```
print("\nAccuracy:")
```

```
print(accuracy_score(y_test, pred_log))
```

```
print("\nMatriz de confusión:")
```

```
print(confusion_matrix(y_test, pred_log))
```

```
print("\nReporte de clasificación:")
```

```
print(classification_report(y_test, pred_log))
```

```
# Coeficientes
```

```
coeficientes = pd.DataFrame({  
    "Variable": X.columns,  
    "Coeficiente": modelo_log.coef_[0]  
})
```

```
print("\nImportancia de variables:")
```

```
print(coeficientes.sort_values(  
    by="Coeficiente",  
    ascending=False  
))
```

```
# =====
```

```
# INTERPRETACIÓN DOCENTE
```

```
# =====
```

```
#
```

```
# Un coeficiente positivo indica que al aumentar la variable
```

```
# incrementa la probabilidad de ataque.
```

```

#

# Si "intentos_fallidos" posee el coeficiente más alto,

# significa que es un indicador crítico de intrusión.

#

# =====

# =====

# PASO 3: CLASIFICACIÓN CON KNN

# =====

print("\n=====")

print("KNN")

print("=====")


k_values = range(1, 21)

accuracy_scores = []


for k in k_values:


    knn = KNeighborsClassifier(n_neighbors=k)


    scores = cross_val_score(

        knn,

```

```
X_train_scaled,

y_train,

cv=5,

scoring='accuracy'

)

accuracy_scores.append(scores.mean())

# Mejor K

best_k = k_values[np.argmax(accuracy_scores)]

print(f"\nMejor valor de K: {best_k}")

print(f"Accuracy CV: {max(accuracy_scores):.4f}")

# Entrenamiento final

modelo_knn = KNeighborsClassifier(

    n_neighbors=best_k

)

modelo_knn.fit(X_train_scaled, y_train)

pred_knn = modelo_knn.predict(X_test_scaled)
```

```
print("\nAccuracy en prueba:")

print(accuracy_score(y_test, pred_knn))

print("\nMatriz de confusión:")

print(confusion_matrix(y_test, pred_knn))
```

```
# Gráfico Accuracy vs K

plt.figure(figsize=(8,5))

plt.plot(k_values, accuracy_scores, marker='o')

plt.xlabel("Número de Vecinos (K)")

plt.ylabel("Accuracy")

plt.title("Selección del Mejor K")

plt.grid(True)

plt.show()
```

```
# =====
```

```
# INTERPRETACIÓN DOCENTE
```

```
# =====
```

```
#
```

```
# El gráfico muestra cómo cambia la precisión al modificar K.
```

```
#
```

```
# K pequeño:
```

```
# - Mayor sensibilidad
```

```
# - Riesgo de sobreajuste
```

```
#
```

```
# K grande:
```

```
# - Modelo más estable
```

```
# - Puede ignorar ataques reales
```

```
#
```

```
# El mejor K es aquel que maximiza la Accuracy.
```

```
#
```

```
# =====
```

```
# =====
```

```
# PASO 4: CLUSTERING K-MEANS
```

```
# =====
```

```
print("\n=====")
```

```
print("K-MEANS")
```

```
print("=====")
```

```
X_cluster = StandardScaler().fit_transform(
```

```
    df_red.drop('es_ataque', axis=1)
```

```
)
```

```
kmeans = KMeans(
```

```
n_clusters=2,  
random_state=111,  
n_init=10  
)
```

```
clusters = kmeans.fit_predict(X_cluster)
```

```
df_red["cluster"] = clusters
```

```
# Comparación con la realidad
```

```
comparacion = pd.crosstab(  
    df_red['es_ataque'],  
    df_red['cluster'],  
    rownames=['Real'],  
    colnames=['Cluster']  
)
```

```
print("\nTabla de comparación:")
```

```
print(comparacion)
```

```
# =====
```

```
# VISUALIZACIÓN DE CLUSTERS
```

```
# =====
```



```
plt.figure(figsize=(8,6))
```

```
sns.scatterplot(  
    data=df_red,  
    x='latencia',  
    y='intentos_fallidos',  
    hue='cluster',  
    palette='Set1'  
)
```

```
plt.title("Clusters Descubiertos por K-Means")
```

```
plt.show()
```

```
# =====
```

```
# INTERPRETACIÓN DOCENTE
```

```
# =====
```

```
#
```

```
# Si un cluster contiene principalmente ataques
```

```
# y el otro conexiones seguras, significa que:
```

```
#
```

```
# - Existe una firma digital clara del comportamiento
```

```
# malicioso.
```

```
#

# - El algoritmo puede detectar patrones anómalos

# incluso sin conocer previamente la etiqueta.

#

# - Esto demuestra el potencial de los métodos

# no supervisados para ciberseguridad.

#

# =====

# =====

# RESUMEN FINAL

# =====

print("\n===== RESUMEN =====")

print(f"Accuracy Regresión Logística: "

      f"{accuracy_score(y_test, pred_log):.4f}")

print(f"Accuracy KNN: "

      f"{accuracy_score(y_test, pred_knn):.4f}")

print(f"Mejor K encontrado: {best_k}")
```

```
print("\nProceso completado correctamente.")
```

```
In [9]: %runfile 'C:/Users/liclo/Documents/Sin título7.py' --wdir
Primeras filas:
   latencia  intentos_fallidos  tamaño_paquete  es_ataque
0  21.787200                2      377.550984        0
1  21.887692                2      611.967019        0
2  26.911689                0      405.019900        0
3  25.877744                1      604.842439        0
4  15.303622                0      531.662593        0

Distribución de clases:
es_ataque
0      369
1       31
Name: count, dtype: int64
```

```
Estadísticas descriptivas:
   latencia  intentos_fallidos  tamaño_paquete  es_ataque
count  400.000000      400.000000      400.000000  400.000000
mean    20.114234        0.987500      492.354915    0.077500
std     5.146370        1.002425       98.573153    0.267718
min     3.484642        0.000000      154.568296    0.000000
25%    16.809845        0.000000      431.088528    0.000000
50%    19.995391        1.000000      493.983972    0.000000
75%    23.734348        1.000000      552.993893    0.000000
max    34.353435        5.000000      770.977113    1.000000
```

```
=====
REGRESIÓN LOGÍSTICA
=====

Accuracy:
1.0

Matriz de confusión:
[[111  0]
 [ 0  9]]

Reporte de clasificación:
   precision  recall  f1-score  support
0          1.00    1.00    1.00     111
1          1.00    1.00    1.00        9

accuracy          1.00     120
macro avg         1.00    1.00    1.00     120
weighted avg      1.00    1.00    1.00     120
```

```
Importancia de variables:
      Variable  Coeficiente
1  intentos_fallidos    3.766006
0      latencia        0.078317
2    tamano_paquete   -0.033555
```

```
=====
KNN
=====
```

```
Mejor valor de K: 1
Accuracy CV: 0.9893
```

```
Accuracy en prueba:
0.9916666666666667
```

```
Matriz de confusión:
[[111  0]
 [ 1  8]]
```

```
=====
K-MEANS
=====
```

```
Tabla de comparación:
```

```
Cluster   0   1
Real
0         218 151
1           6  25
```

```
C:\Users\liclo\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=2.
  warnings.warn(
```

```
===== RESUMEN =====
Accuracy Regresión Logística: 1.0000
Accuracy KNN: 0.9917
Mejor K encontrado: 1
```

```
Proceso completado correctamente.
```